
Guía de Estudio: Sistemas Operativos

I. Introducción a los Sistemas Operativos

- **Definición y objetivos:**

Un sistema operativo (SO) es el software que actúa como interfaz entre el usuario y el hardware, gestionando recursos, coordinando tareas y asegurando la seguridad y eficiencia del sistema.

- **Funciones generales:**

- Proveer una interfaz (por ejemplo, Shell) para la interacción con el usuario.
- Gestionar operaciones de entrada/salida y el almacenamiento (memoria principal y secundaria).
- Coordinar la ejecución de procesos y asignar recursos (CPU, memoria, etc.).
- Implementar mecanismos de seguridad, protección de archivos y recuperación de errores.

(Ver detalles en y)

II. Componentes y Módulos Principales

1. Soporte de Hardware y Mecanismos Básicos

- **Interrupciones:**

- Señales que interrumpen la ejecución del proceso actual para atender eventos mediante una rutina de servicio (ISR).
- Proceso: guardar el estado, identificar la interrupción, ejecutar la ISR y retomar la actividad anterior.

()

- **Llamadas al Sistema (System Calls):**

- Permiten que las aplicaciones soliciten servicios que solo el kernel puede ejecutar.

()

- **Timer:**

- Genera interrupciones a intervalos predefinidos para gestionar el tiempo de ejecución de procesos.

()

- **Mode Bit (Bit de Modo):**

- Diferencia entre el **modo kernel** (nivel de privilegio para ejecutar operaciones críticas) y el **modo usuario** (nivel restringido para aplicaciones), previniendo que programas afecten el funcionamiento del SO.
()

2. Gestión de Procesos

- **Definición de proceso:**

Un proceso es un programa en ejecución que requiere administración para su creación, eliminación, suspensión, y sincronización.

- **Funciones clave:**

- Crear y eliminar procesos.
- Suspende y reanuda su ejecución.
- Sincronización y comunicación entre procesos. (*Información complementaria en*)

3. Gestión de Memoria y Almacenamiento

- **Memoria Principal y Secundaria:**

- La memoria principal (RAM) almacena los programas en ejecución, mientras que la secundaria (discos, USB, etc.) guarda datos y archivos a largo plazo.

- **Asignación y liberación:**

- Distribución de espacios de memoria entre procesos y recuperación cuando ya no son necesarios. ()

4. Gestión de Archivos

- **Concepto de archivo:**

Una colección de bytes que pueden representar datos, programas, imágenes, etc.

- **Operaciones:**

- Creación, manipulación, borrado y respaldo de archivos y directorios. ()

5. Gestión de Dispositivos de Entrada/Salida (E/S)

- **Drivers y abstracción del hardware:**

- Los controladores (drivers) permiten que el SO oculte las particularidades del hardware, facilitando la instalación y el uso de dispositivos. ()

6. Gestión de Redes y Protocolos

- **Administración de la red:**

- Configuración y gestión de conexiones entre computadoras para compartir recursos.

- **Protocolos y herramientas:**

- Protocolos: DHCP, HTTP, FTP, SSH, entre otros.

- Herramientas de conexión como **Putty**, que permite conexiones SSH o telnet para la administración remota.
- Diferencias clave:
 - **Telnet**: Envía datos (incluyendo contraseñas) sin cifrado.
 - **SSH**: Proporciona una conexión segura con cifrado. (*Detalles en*

7. Gestión de la Seguridad

- **Objetivos de la seguridad:**
 - Prevenir que procesos interfieran en otros, protegiendo la integridad del sistema y evitando vulnerabilidades (virus, ataques de denegación de servicio, etc.).
 - **Mecanismos:**
 - Aislamiento de procesos, control de acceso a recursos y multiplexación de tiempo para garantizar la equidad en el uso del CPU. ()
-

III. Organización Modular del Sistema Operativo

Los sistemas operativos suelen organizarse en módulos, cada uno especializado en una función clave:

- **Manejo de Memoria:** Asignación y seguimiento del uso de la RAM.
- **Manejo de E/S:** Coordinación de la comunicación con dispositivos y periféricos.
- **Manejo del Sistema de Archivos:** Organización y protección de datos en dispositivos de almacenamiento.
- **Manejo de Procesos:** Control y coordinación de la ejecución de programas.

(Conceptos generales extraídos de y)

IV. Sistemas Operativos Basados en Unix

Características Generales

- **Multiusuario y Multitarea:**
 - Permiten que varios usuarios utilicen el sistema simultáneamente, gestionando múltiples procesos a la vez.
- **Arquitectura de Kernel y Shell:**
 - El **kernel** administra los recursos y la comunicación con el hardware, mientras que el **shell** actúa como intérprete de comandos entre el usuario y el sistema.
- **Ejemplos y variantes:**
 - UNIX, MULTICS, UNICS.

Tipos de Shells en Unix

- **Principales shells:**
 - Korn Shell, Bourne Shell, C Shell, Bourne-Again Shell (BASH).
- **Aspectos a considerar:**
 - Características, interfaz (teclado o mouse), funcionalidades, disponibilidad, y el grupo de usuarios al que están dirigidos.

(Información adicional en)

V. Herramientas y Protocolos de Conexión en Redes

Uso de Aplicaciones de Conexión Remota

- **Putty y herramientas similares:**
 - Permiten conexiones mediante SSH o telnet para administrar servidores de forma remota.
- **Configuración básica con Putty:**
 - Se requiere especificar la IP del servidor, el puerto correspondiente (por ejemplo, 22 para SSH, 20/21 para FTP) y otras configuraciones como la autenticación.
- **Protocolos relevantes:**
 - **HTTP/HTTPS:** Para la comunicación con servidores web.
 - **FTP/vsftp:** Para la transferencia de archivos.
 - **SSH/Telnet:** Para conexiones seguras (o menos seguras, en el caso de Telnet) a dispositivos remotos.

(Detalles prácticos en)

VI. Consideraciones Adicionales para el Estudio

- **Sistemas Multiusuario vs. Sistemas de Red:**
 - En un sistema multiusuario, varios usuarios comparten un único procesador y recursos a través de terminales conectadas a un servidor central, mientras que en una red, cada máquina cuenta con su propio procesador y recursos.
 - **Preguntas de Exámenes (Ceneval):**
 - Repasar el funcionamiento del modelo OSI y la configuración de servicios (como DHCP, HTTP, FTP, SSH, entre otros) en el contexto de una red.
 - **Comandos y Prácticas de Consola:**
 - Familiarizarse con comandos básicos (por ejemplo, `$ pwd`, `$ passwd`) y la navegación en el entorno de un SO.
-

VII. Recomendaciones Finales

- **Repaso Teórico y Práctico:**
 - Es importante complementar la teoría con prácticas reales: configurar conexiones remotas, trabajar en la administración de procesos y gestionar archivos y memoria.
 - **Ejercicios y Casos Prácticos:**
 - Realizar ejercicios que impliquen la utilización de comandos de shell, el manejo de interrupciones y la configuración de servicios en red.
 - **Consulta de Fuentes Adicionales:**
 - Revisar documentación y manuales de referencia (como manuales de Unix/Linux) para profundizar en los temas tratados.
-

1. Diferencia entre > y >>

> (sobrescritura): redirige la salida estándar de un comando a un archivo, sobrescribiendo su contenido si el archivo ya existe.

>> (adición): redirige la salida estándar a un archivo añadiéndola al final, sin sobrescribir el contenido previo.

2. ¿Qué es Bash, Ksh, etc.?

Shell: Es un intérprete de comandos en sistemas Unix/Linux.

Bash (Bourne Again Shell): El shell más común en Linux; compatible con muchos scripts y ofrece características como autocompletado, historial avanzado, etc.

Ksh (KornShell): Shell más antiguo y muy usado en entornos Unix tradicionales; destaca por sus funcionalidades de scripting.

Otros shells (Zsh, Csh, Tcsh, etc.): Cada uno tiene sus propias características y sintaxis ligeramente diferentes.

3. Módulos del Sistema Operativo que se pueden omitir y por qué

Los módulos son componentes (drivers, funcionalidades adicionales) que pueden cargarse o descargarse según las necesidades del sistema.

Razones para omitir módulos:

Reducir el tamaño del kernel (especialmente en sistemas embebidos).

Evitar posibles vulnerabilidades en módulos que no se utilizan.

Optimizar el rendimiento al no cargar controladores innecesarios.

4. Cómo afecta la arquitectura de 8, 16, 32 (y 64) bits a la memoria RAM

El “número de bits” de una arquitectura está relacionado con:

El tamaño de los registros internos del procesador.

La capacidad de direccionamiento de memoria.

8 bits: puede direccionar hasta $2^8 = 256$ direcciones.

16 bits: hasta $2^{16} = 65.536$ direcciones (64 KB).

32 bits: hasta $2^{32} \approx 4.294.967.296$ direcciones (4 GB).

64 bits: hasta 2^{64} , un espacio de direcciones muchísimo mayor.

Impacto principal: Cuanta más amplitud de bits tenga la CPU, más memoria se puede utilizar directamente y mejor se gestionan operaciones con números grandes.

5. Módulo de seguridad en el SO: para qué sirve

Un módulo de seguridad (ej. SELinux, AppArmor) es un componente que:

Controla el acceso a los recursos del sistema.

Define y hace cumplir políticas de seguridad (quién puede hacer qué).

Ayuda a aislar procesos y proteger la integridad del sistema.

6. Funcionalidad del Sistema Operativo

El SO es la capa de software que interactúa entre el hardware y las aplicaciones. Sus funciones principales incluyen:

Gestión de procesos: creación, planificación, sincronización y terminación de procesos.

Gestión de memoria: asignación, intercambio, paginación y protección de la memoria.

Gestión de dispositivos (drivers): comunicación y control de periféricos.

Sistema de archivos: creación, lectura, escritura y organización de archivos y directorios.

Seguridad y control de accesos: autenticación de usuarios, permisos, etc.

Interfaz de usuario: ya sea línea de comandos o entorno gráfico.

7. Cómo ver un inodo

En Linux, puedes ver el número de inodo de un archivo con el comando `ls -li`.

¿Qué es un inodo?

Una estructura de datos que contiene metadatos sobre un archivo: permisos, dueño, tamaño, fechas (creación, modificación), ubicación en disco, etc.

No almacena el nombre del archivo; los nombres se guardan en directorios que apuntan al inodo.

8. Diferencia entre una liga dura (hard link) y cp

Liga dura (hard link):

Crea una nueva referencia (nombre) al mismo inodo en el mismo sistema de archivos.

No ocupa espacio adicional (más allá de la referencia).

Si se borra el archivo “original”, los datos siguen existiendo mientras exista al menos un hard link apuntando al inodo.

cp:

Crea una copia independiente con un inodo diferente.

Ocupa espacio adicional.

Si se borra el archivo original, la copia permanece intacta e independiente.

9. Diferencias entre cut, sort y grep

cut:

Extrae secciones de cada línea (campos delimitados por comas, tabulaciones o posiciones de carácter).

sort:

Ordena las líneas de texto según diferentes criterios (alfabético, numérico, inverso, etc.).

grep:

Busca líneas que coincidan con un patrón (expresiones regulares).

Cuándo usar cada uno:

cut para extraer columnas o partes de un archivo de texto.

sort para reordenar listas o resultados de otros comandos.

grep para filtrar o encontrar patrones específicos.

10. Diferencia entre CISC y RISC

CISC (Complex Instruction Set Computing):

Conjunto de instrucciones amplio y complejo.

Cada instrucción puede realizar múltiples operaciones.

Ejemplo clásico: la familia x86 (Intel/AMD).

Ventaja: Menos líneas de código ensamblador.

Desventaja: Hardware más complejo, consumo de energía mayor.

RISC (Reduced Instruction Set Computing):

Conjunto de instrucciones reducido y más simple.

Mayor énfasis en la velocidad y la ejecución en pipeline.

Ejemplo: ARM, MIPS, PowerPC.

Ventaja: Procesadores más eficientes, menores consumos de energía, gran rendimiento en pipelines.

Desventaja: A veces más instrucciones a nivel ensamblador para realizar tareas complejas.

11. ¿De qué dispositivo es la interrupción más importante?

A menudo se considera que la interrupción de reloj (timer) es crucial, porque:

Permite al sistema operativo gestionar la multitarea, asignando tiempo de CPU a los procesos.

Sin esta interrupción periódica, un proceso podría monopolizar la CPU.

Otras interrupciones importantes son las de dispositivos de entrada/salida (teclado, disco, red, etc.), pero la de reloj es fundamental para el correcto funcionamiento del planificador de procesos.

12. Cómo funciona una interrupción

Un dispositivo o evento (teclado, red, temporizador, etc.) envía una señal de interrupción al procesador.

El procesador detiene temporalmente la ejecución del proceso actual y guarda su estado (registros, contador de programa).

Se transfiere el control a la rutina de servicio de interrupción (ISR) correspondiente.

Al terminar el servicio de interrupción, se restaura el estado anterior y se retoma la ejecución donde se interrumpió.

13. Diferencia entre micronúcleo y monolítico

Núcleo monolítico:

Todo (gestión de memoria, drivers, sistema de archivos, etc.) se ejecuta en modo kernel.

Ejemplo: Linux.

Ventajas: Mejor rendimiento al haber menos cambios de contexto.

Desventajas: Un fallo en un módulo puede afectar a todo el sistema. Menos modular.

Micronúcleo (microkernel):

Se deja en modo kernel solo lo esencial (planificación de procesos, comunicación entre procesos, etc.).

Otros servicios (drivers, sistemas de archivos) se ejecutan en espacio de usuario.

Ejemplo: Minix, QNX.

Ventajas: Más estabilidad y seguridad, porque cada servicio está aislado.

Desventajas: Mayor sobrecarga en la comunicación entre servicios, lo que puede impactar en el rendimiento.

Consejos de estudio

Practica con ejemplos reales en un sistema Linux: crea archivos, haz redirecciones, prueba diferentes shells.

Utiliza manuales (man command) para profundizar en cut, sort, grep y otros comandos.

Revisa documentación sobre arquitecturas de CPU (CISC/RISC) y tipos de núcleos de SO (monolítico vs micronúcleo) para entender mejor sus implicaciones prácticas.

Observa cómo el sistema operativo maneja interrupciones: revisa /proc/interrupts en Linux para ver estadísticas de interrupciones.

Guía de Estudio Detallada – Taller de Sistema Operativo Unix

Esta guía recopila los conceptos, estructuras y comandos fundamentales que se abordan en el manual de prácticas de Unix, distribuidos en tres grandes secciones: la introducción y fundamentos de Unix, la organización y manejo del sistema de archivos, y la administración de archivos y permisos. Se cubren las páginas 2 a 8 y 10 a 13.

I. Introducción y Fundamentos de Unix (Páginas 2-4)

1. Concepto y Funciones de un Sistema Operativo

- **Definición:**

Un sistema operativo es el software encargado de gestionar los recursos de la computadora y actuar como interfaz entre el usuario y el hardware.

- **Funciones Esenciales:**

- **Interfaz Usuario-Máquina:** Permite al usuario comunicarse con el sistema.
- **Gestión del Tiempo:** Lee y actualiza la fecha y hora, aprovechando el reloj integrado en el hardware.
- **Operaciones de Entrada/Salida (E/S) y Organización del Disco:** Coordina el acceso a dispositivos y organiza los datos en disco.
- **Acceso a Dispositivos:** Facilita la comunicación con periféricos (impresoras, terminales, etc.).
- **Protección y Compartición de Datos:** Implementa mecanismos para salvaguardar archivos y posibilitar el trabajo en red.
- **Provisión de Herramientas y Recuperación de Errores:** Ofrece utilidades y soporta la detección y corrección de fallos.
- **Asignación de Recursos:** Distribuye de manera equitativa memoria, espacio en disco y tiempo de procesamiento entre los usuarios.

2. Organización Modular del Sistema Operativo

- **Módulos Principales:**

- **Manejo de Memoria:** Controla la asignación y liberación de la memoria.
- **Manejo de E/S:** Administra la comunicación entre la computadora y los dispositivos externos.
- **Manejo del Sistema de Archivos:** Organiza y protege los datos en disco.
- **Manejo de Procesos:** Supervisa y gestiona la ejecución concurrente de programas.

3. Introducción a Unix y su Historia

- **Orígenes:**

- Unix fue uno de los primeros sistemas operativos escritos en un lenguaje de alto nivel y fue desarrollado en los laboratorios Bell a finales de los años 60.

- **Distribución y Evolución:**

- Inicialmente, Unix se distribuyó a bajo costo a universidades y centros de investigación. En 1980, AT&T obtuvo libertad para comercializarlo, lo que propició su expansión en el mundo de los negocios.

- **Características Clave:**

- **Multiusuario:** Permite que varios usuarios utilicen el sistema al mismo tiempo, manteniendo la privacidad de los archivos personales.

- **Multitarea:** Capacidad del procesador de ejecutar múltiples tareas simultáneamente.
- **Multiproceso:** Soporta el trabajo con varios procesadores, lo que permite la ejecución concurrente de instrucciones.

4. Estructura Básica de Unix: Kernel y Shell

- **Kernel:**
 - Es el núcleo del sistema operativo. Entre sus funciones se incluyen:
 - Gestión de la memoria.
 - Control de accesos y seguridad.
 - Mantenimiento del sistema de archivos.
 - Manejo de interrupciones y errores.
 - Provisión de servicios de entrada/salida.
 - Asignación de recursos de hardware.
- **Shell:**
 - Es el intérprete de comandos que actúa como interfaz entre el usuario y el kernel.
 - **Funciones del Shell:**
 - Recibe y analiza la línea de comandos introducida por el usuario.
 - Traduce los comandos y los envía al kernel para su ejecución.
 - Facilita características como autocompletado, historial de comandos y ejecución de secuencias de instrucciones (scripts).
 - **Ejemplos de Shells:**
 - **Bourne Shell:** El shell original en Unix, desarrollado por Stephen Bourne.
 - **Korn Shell:** Desarrollado por David Korn, incorpora mejoras en la sintaxis.
 - **C Shell:** Diseñado con sintaxis similar al lenguaje C, popular entre programadores.
 - **Bourne-Again Shell (bash):** Muy utilizado en sistemas modernos por su compatibilidad y facilidad de uso.

5. Proceso de Inicio de Sesión en Unix

- **Secuencia de Arranque:**
 - Al encender la máquina, el proceso **init** inicia la configuración del sistema.
 - Se ejecuta **getty** para establecer la comunicación en cada terminal, mostrando la cadena de login.
 - El programa **login** verifica las credenciales del usuario (nombre y password).
 - Una vez autenticado, se lanza el **shell** (por ejemplo, **sh** o **bash**) que ejecuta el archivo de configuración (como **.profile**) y presenta el prompt para la entrada de comandos.
- **Cuentas de Usuario y Roles:**
 - **Login Name y Password:** Identifican al usuario.

- **Group Identification:** Cada usuario pertenece a uno o más grupos, lo cual influye en el acceso a ciertos archivos.
 - **Home Directory:** Directorio personal donde se almacenan los archivos del usuario.
 - **Superusuario (root):** Posee privilegios especiales para administrar todo el sistema.
-

II. Sistema de Archivos en Unix (Páginas 5-8)

1. Concepto de Archivo y Organización Jerárquica

- **Definición de Archivo:**
 - Un archivo es una sucesión de bytes que puede representar datos, programas o dispositivos.
- **Organización del Sistema de Archivos:**
 - Se organiza en una estructura de árbol (jerárquica) en la que el directorio raíz se denota con `/`.
 - Cada archivo y directorio es representado internamente por un **i-nodo**, que almacena información crítica como:
 - Fecha de modificación.
 - Propietario y grupo.
 - Tamaño en bytes.
 - Permisos de acceso.

2. Directorios Estándar y su Función

- **Directorio Raíz (/):**
 - Es el punto de partida de toda la estructura del sistema.
- **Subdirectorios Importantes:**
 - **/bin:** Contiene los comandos básicos de usuario.
 - **/sbin:** Aloja comandos esenciales para el superusuario.
 - **/usr:** Reúne aplicaciones e información del sistema, incluyendo:
 - **/usr/bin:** Comandos de usuario.
 - **/usr/local:** Programas y archivos instalados específicamente en la máquina.
 - **/usr/lib:** Librerías para aplicaciones.
 - **/usr/doc** y **/usr/man:** Documentación y páginas de manual.
 - **/lib:** Librerías del sistema.
 - **/etc:** Archivos de configuración.
 - **/dev:** Archivos que representan dispositivos físicos.
 - **/var:** Archivos de datos variables, como logs y colas de impresión.
 - **/tmp:** Archivos temporales.
 - **/proc:** Información sobre procesos en ejecución.
 - **/home:** Directorios personales de los usuarios.

3. Conceptos de Rutas y Notación de Directorios

- **Directorio Actual y Padre:**
 - `.` representa el directorio en el que se está trabajando.
 - `..` indica el directorio padre.
 - `/` es la raíz del sistema.
- **Rutas Absolutas vs. Relativas:**
 - **Ruta Absoluta:** Comienza con `/` y especifica la localización completa (por ejemplo, `/export/home/maestro/fuente.txt`).
 - **Ruta Relativa:** Se basa en el directorio de trabajo actual (por ejemplo, `./fuente.txt` o `../home/maestro/fuente.txt`).

4. Comandos Básicos para el Manejo de Directorios

- **Visualización de Contenido:**
 - `ls`: Lista el contenido de un directorio.
 - Opciones comunes:
 - `-l`: Listado detallado (permite ver permisos, propietario, tamaño y fecha).
 - `-a`: Muestra archivos ocultos (aquellos cuyo nombre comienza con un punto).
 - `-F`: Añade un símbolo al final del nombre (por ejemplo, `/` para directorios, `*` para ejecutables).
 - `-R`: Lista recursivamente subdirectorios.
- **Navegación y Gestión:**
 - `pwd`: Muestra el directorio de trabajo actual.
 - `cd`: Permite cambiar de directorio.
 - Ejemplo: `cd ..` para regresar al directorio padre o `cd nombre_directorio` para ingresar a uno específico.
 - `mkdir`: Crea un nuevo directorio.
 - Ejemplo: `mkdir tareas` crea un directorio llamado “tareas”.
 - `mv`: Mueve o renombra directorios y archivos.
 - Ejemplo: `mv antiguo nuevo` cambia el nombre o mueve el objeto.
 - `rmdir`: Elimina un directorio vacío.
- **Comparativa con DOS:**
 - Muchos comandos de Unix tienen equivalentes en DOS, pero la sintaxis y opciones pueden variar (por ejemplo, `ls` en Unix frente a `dir` en DOS).

III. Administración de Archivos y Permisos (Páginas 10-13)

1. Definición y Atributos de un Archivo

- **Qué es un Archivo:**
 - Es una colección de bytes que puede almacenar datos, programas o información de dispositivos.
- **Atributos Principales:**
 - **Nombre:** Identificador del archivo (debe reflejar su contenido).
 - **I-nodo:** Estructura de datos interna que almacena información como fecha de modificación, tamaño y permisos.
 - **Tamaño:** Número de bytes que contiene.
 - **Fecha de Última Modificación:** Momento en que el archivo fue alterado por última vez.
 - **Permisos:** Reglas de acceso para el dueño, grupo y otros usuarios.
 - **Propietario y Grupo:** El usuario dueño del archivo y el grupo al que pertenece.

2. Tipos de Archivos en Unix

- **Archivos Ordinarios:**
 - Pueden ser documentos, código fuente o programas ejecutables.
 - Su contenido puede interpretarse como texto o datos binarios.
- **Archivos de Dispositivo:**
 - Representan dispositivos físicos (como discos duros, impresoras, terminales).
- **Archivos de Directorios:**
 - Son contenedores que agrupan archivos. Contienen los nombres y ubicaciones de los archivos “almacenados” en ellos.

3. Nombres de Archivos y Uso de Comodines

- **Nombres de Archivos:**
 - Deben ser descriptivos y únicos dentro de un mismo directorio, aunque pueden repetirse en directorios diferentes.
 - Los archivos que comienzan con un punto (.) se consideran ocultos, lo que es común para archivos de configuración.
- **Comodines en Unix:**
 - *****: Representa cualquier secuencia de caracteres (por ejemplo, ***.txt** para todos los archivos de texto).
 - **?**: Coincide con un solo carácter (por ejemplo, **a?c** puede coincidir con “abc” o “axc”).
 - **[]**: Permite definir un conjunto o rango de caracteres (por ejemplo, **[A-C]archivo** coincide con “Aarchivo”, “Barchivo” o “Carchivo”).

4. Comandos para Manejo y Administración de Archivos

- **Visualización del Contenido:**

- **cat**: Muestra el contenido de uno o varios archivos; también se puede usar para concatenar archivos.
- **more** y **less**: Permiten visualizar archivos largos paginando la información para facilitar la lectura.
- **nl**: Muestra el contenido de un archivo numerando sus líneas.
- **Creación y Modificación:**
 - **touch**: Actualiza la fecha de modificación de un archivo o crea uno nuevo si no existe.
 - **mv**: Renombra o mueve archivos a otra ubicación.
 - **cp**: Copia archivos de una ubicación a otra, pudiendo usar opciones interactivas (como **-i**) para confirmar sobrescrituras.
- **Conteo y Extracción de Información:**
 - **wc**: Cuenta líneas, palabras y caracteres de un archivo.
 - **head**: Muestra las primeras líneas de un archivo (por defecto las primeras 10).
 - **tail**: Muestra las últimas líneas (por defecto las últimas 10, pero se puede ajustar con **-n**).

5. Permisos de Archivos y Directorios

- **Concepto de Permisos:**
 - Cada archivo o directorio tiene asignados permisos para tres categorías:
 - **Dueño (u)**
 - **Grupo (g)**
 - **Otros (o)**
 - Los permisos básicos son:
 - **r (read)**: Permite leer el contenido.
 - **w (write)**: Permite modificar o borrar el contenido.
 - **x (execute)**: Permite ejecutar un archivo o “entrar” en un directorio.
- **Significado Específico para Directorios:**
 - **r**: Permite listar los archivos contenidos.
 - **w**: Permite crear, borrar o renombrar archivos dentro del directorio.
 - **x**: Permite acceder o buscar dentro del directorio.
- **Cambiar Permisos con **chmod**:**
 - **Método Absoluto (Numérico):**
 - Se asigna un número octal de tres dígitos a cada categoría (por ejemplo, 750 asigna al dueño permisos completos [7 = 4+2+1], al grupo permisos de lectura y ejecución [5 = 4+0+1] y a otros ningún permiso [0]).
 - Ejemplo: **chmod 750 pract1**
 - **Método Simbólico:**
 - Se utiliza una combinación de letras y operadores para añadir (+), quitar (-) o establecer permisos exactos (=).
 - Ejemplos:

- `chmod u+x archivo` (Añade permiso de ejecución al dueño).
- `chmod go+x archivo` (Añade permiso de ejecución al grupo y a otros).
- `chmod o-w archivo` (Quita permiso de escritura a otros).
- `chmod a=r archivo` (Establece que todos tengan solo permiso de lectura).